

π -games: Normal-Form Games in Typst

Piotr Kuszewski April 2026

1 Introduction

The `pi-games` library provides two Typst functions for typesetting *normal-form* (strategic-form) games of the kind common in game theory courses and research:

game-normal-form draws a two-player $N \times M$ payoff matrix. Each cell displays both players' payoffs separated by a comma, coloured by player.

game-three-player-normal-form draws a three-player game as a collection of $N \times M$ matrices — one per strategy of Player 3 — laid out in rows.

Both functions are built on the `@preview/cetz` drawing package. They measure content at layout time and automatically size each cell to fit its widest payoff expression or strategy label, so the typesetter never needs to specify column widths by hand.

Import the library at the top of your document:

```
1 #import "pi-games.typ": *
```

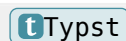


2 game-normal-form

2.1 Payoff Array Structure

The `payoffs` argument is a 2-dimensional array. Element `payoffs.at(r).at(c)` holds a 2-element array $(v1, v2)$ where $v1$ is Player 1's payoff and $v2$ is Player 2's payoff when Player 1 plays row r and Player 2 plays column c . Both values are ordinary Typst content and may contain arbitrary markup or mathematics.

```
1 payoffs: (  
2   // row 0  
3   ( (v1_00, v2_00), (v1_01, v2_01) ),  
4   // row 1  
5   ( (v1_10, v2_10), (v1_11, v2_11) ),  
6 )
```



2.2 Auto-Sizing

Cell width is determined automatically at render time as the maximum of three quantities:

1. **cell-width**: the user-supplied lower bound (defaults to `game-cell-width`, initially 5em).

2. **Strategy-label width:** the rendered width of the widest Player 2 strategy label plus 2em of horizontal padding. This ensures each column is never narrower than its header.
3. **Payoff width:** the rendered width of the widest payoff pair v_1, v_2 — including best-response underlines where active — plus 2em of padding.

The measurement happens inside a **context** block, so it uses the exact font metrics of every piece of content, including mathematical formulas of arbitrary complexity. The **left margin** (Player 1 strategy labels and rotated player name) is auto-sized to the widest Player 1 label; the **top margin** (Player 2 strategy labels and player name) is auto-sized to the tallest Player 2 label. **Cell height** is fixed at **cell-height** (defaults to **game-cell-height**, initially 2em) and is not auto-sized, since all payoff content is placed on a single line.

2.3 Best Responses and Nash Equilibria

- **p1-best** / **p2-best:** arrays of (row, col) tuples. In each listed cell the respective player's payoff is underlined with a 1 pt stroke in their colour (**game-pal.at(0)** and **game-pal.at(1)**).
- **nash:** array of (row, col) tuples. A coloured rectangle in **game-nash-color** is drawn just inside the cell border.

Both underlines and Nash rectangles are taken into account when auto-sizing, so highlighting never causes payoffs to overflow their cells.

2.4 Examples

2.4.1 Prisoner's Dilemma

The Prisoner's Dilemma is the textbook example of a dominant-strategy equilibrium. Defection strictly dominates Cooperation for both players, so (D,D) is the unique Nash equilibrium even though (C,C) Pareto-dominates it.

```

1  #game-normal-form(
2    [Prisoner 1], [Prisoner 2],
3    ([C], [D]),
4    ([C], [D]),
5    (
6      ([[3$], [3$]], [[0$], [5$]]),
7      ([[5$], [0$]], [[1$], [1$]]),
8    ),
9    p1-best: ((1, 0), (1, 1)),
10   p2-best: ((0, 1), (1, 1)),
11   nash: ((1, 1)),
12 )

```

		Prisoner 2	
		C	D
Prisoner 1	C	3, 3	0, 5
	D	5, 0	1, 1

2.4.2 Battle of Sexes

In the Battle of Sexes the players want to coordinate but disagree on which outcome to coordinate on. There are two pure-strategy Nash equilibria — both choose Opera, or both choose Football — and a mixed-strategy equilibrium in between.

```

1 #game-normal-form(
2   [She], [He],
3   ([Opera], [Football]),
4   ([Opera], [Football]),
5   (
6     ([[2$]], [[1$]], ([[0$]], [[0$]]),
7     ([[0$]], [[0$]], ([[1$]], [[2$]]),
8   ),
9   p1-best: ((0, 0), (1, 1)),
10  p2-best: ((0, 0), (1, 1)),
11  nash: ((0, 0), (1, 1)),
12 )

```

		He	
		Opera	Football
She	Opera	2, 1	0, 0
	Football	0, 0	1, 2

2.4.3 Mixed-Strategy Equilibrium: Matching Pennies

Strategy labels are arbitrary Typst content, so mixed-strategy equilibrium probabilities can be embedded directly in the label. Matching Pennies has no pure-strategy Nash equilibrium; the unique Nash equilibrium requires each player to play Heads with probability $p^* = q^* = \frac{1}{2}$. Including $[p]$ and $[q]$ in the strategy names makes the equilibrium condition explicit. The column width auto-adjusts to accommodate the wider labels.

```

1 #game-normal-form(
2   [P1], [P2],
3   ([H #h(1mm) $[p]$], [T #h(1mm) $[1-p]$]),
4   ([H $[q]$], [T $[1-q]$]),
5   (
6     ([[1$]], [[-1$]], [[-1$]], [[1$]]),
7     ([[1$]], [[-1$]], [[1$]], [[-1$]]),

```

```
8   ),
9   )
```

		P2	
		$[q]$ H	$[1 - q]$ T
P1	$[p]$ H	1, -1	-1, 1
	$[1 - p]$ T	-1, 1	1, -1

2.4.4 Parametric Payoffs: 3×3 Coordination Game

Payoff values may contain mathematical variables and expressions of any complexity. Auto-sizing measures the actual rendered width of each expression to determine the cell size. The symmetric 3×3 coordination game has payoff a when players match and b otherwise; all three pure-strategy Nash equilibria (A,A) , (B,B) , (C,C) exist for any $a > b$.

```
1  #game-normal-form(
2    [P1], [P2],
3    ([A], [B], [C]),
4    ([A], [B], [C]),
5    (
6      ([[a$], [a$]], [[b$], [b$]], [[b$], [b$]]),
7      ([[b$], [b$]], [[a$], [a$]], [[b$], [b$]]),
8      ([[b$], [b$]], [[b$], [b$]], [[a$], [a$]]),
9    ),
10   p1-best: ((0, 0), (1, 1), (2, 2)),
11   p2-best: ((0, 0), (1, 1), (2, 2)),
12   nash:    ((0, 0), (1, 1), (2, 2)),
13   )
```

		P2		
		A	B	C
P1	A	a, a	b, b	b, b
	B	b, b	a, a	b, b
	C	b, b	b, b	a, a

3 game-three-player-normal-form

3.1 Payoff Array Structure

The `payoffs` argument is a 3-dimensional array. Element `payoffs.at(k).at(r).at(c)` holds a 3-element array (v_1 , v_2 , v_3):

k (**outermost**) index of Player 3's strategy.
r index of Player 1's strategy (row).
c index of Player 2's strategy (column).

```

1  payoffs: (
2      // k = 0  (first Player 3 strategy)
3      (
4          ((v1, v2, v3), (v1, v2, v3)), // r = 0
5          ((v1, v2, v3), (v1, v2, v3)), // r = 1
6      ),
7      // k = 1  (second Player 3 strategy)
8      (
9          ((v1, v2, v3), (v1, v2, v3)),
10         ((v1, v2, v3), (v1, v2, v3)),
11     ),
12 )

```

Best-response and Nash coordinates use 3-element tuples (**k**, **row**, **col**) throughout.

3.2 Layout and Auto-Sizing

Sub-matrices are placed left to right in rows of at most **games-per-row** sub-matrices (default 2). If Player 3 has more strategies than fit in a row, additional rows are added below. A lone sub-matrix in the last row is horizontally centred within the full row width.

All sub-matrices share identical cell dimensions, computed once from the entire payoff array: the function scans every payoff triple in every cell and every sub-matrix to find the widest rendered content, then combines that with the widest Player 2 strategy label to determine cell width.

Above each sub-matrix (from bottom to top): Player 2's strategy labels, then Player 2's name in bold, then Player 3's strategy label in bold. Player 1's name and strategy labels appear once per row of sub-matrices, to the left. Player 3's player name appears once above the first row of sub-matrices.

3.3 Example: Three-Player Stag Hunt

All three players must cooperate to catch the stag (payoff 3). A player who deviates and hunts the hare alone gets 1; players who pursue the stag without all partners cooperating get 0. Hunting hare is always safe, so (H,H,H) is always a Nash equilibrium; (S,S,S) is the other.

```

1  #game-three-player-normal-form(
2      [P1], [P2], [P3],
3      ([S], [H]),
4      ([S], [H]),
5      ([S], [H]),
6      (

```

```

7      // P3 plays S
8      (
9          ([[3],[3],[3]], ([0],[1],[0])),
10         ([[1],[0],[0]], ([1],[1],[0])),
11     ),
12     // P3 plays H
13     (
14         ([[0],[0],[1]], ([0],[1],[1])),
15         ([[1],[0],[1]], ([1],[1],[1])),
16     ),
17 ),
18 nash:  ((0,0,0), (1,1,1)),
19 p1-best: ((0,0,0), (0,1,1), (1,1,0), (1,1,1)),
20 p2-best: ((0,0,0), (0,1,1), (1,0,1), (1,1,1)),
21 p3-best: ((0,0,0), (1,0,1), (1,1,0), (1,1,1)),
22 )

```

		P3			
		S		H	
		P2		P2	
		S	H	S	H
P1	S	<u>3</u> , <u>3</u> , <u>3</u>	0, 1, 0	0, 0, 1	0, <u>1</u> , <u>1</u>
	H	1, 0, 0	<u>1</u> , <u>1</u> , 0	<u>1</u> , 0, <u>1</u>	<u>1</u> , <u>1</u> , <u>1</u>

4 Global Configuration

All constants are ordinary let bindings. Override them by redefining the binding *after* importing the library:

```

1  #import "pi-games.typ": *
2  #let game-pal = (          // custom player colours
3      rgb("#005f73"), rgb("#94d2bd"), rgb("#e9d8a6"),
4      rgb("#ee9b00"), rgb("#ae2012"),
5  )
6  #let game-nash-color = rgb("#ff6b6b") // custom Nash highlight
7  #let game-fg         = rgb("#222222") // custom foreground
8  #let game-cell-width  = 6em           // wider cells
9  #let game-cell-height = 2.5em        // taller cells
10 #let game-games-per-row = 3           // three sub-matrices per row

```

Name	Default	Role
game-pal	5-colour list	Player colours (index 0 = Player 1, 1 = Player 2, 2 = Player 3).
game-nash-color	rgb("#22c4c7")	Nash equilibrium cell outline colour.
game-fg	rgb("#111111")	Cell borders, comma separators, and punctuation.
game-cell-width	5em	Minimum payoff cell width; auto-grown to fit content.
game-cell-height	2em	Fixed payoff cell height; increase for multi-line content.
game-games-per-row	2	Maximum sub-matrices per row in three-player games.

API Reference

- `game-normal-form()`
- `game-three-player-normal-form()`

Variables

- `game-pal`
- `game-nash-color`
- `game-fg`
- `game-cell-width`
- `game-cell-height`
- `game-games-per-row`

game-normal-form

Draws a general $N \times M$ normal form game table with colored payoffs, best-response underlines, and Nash equilibrium highlights.

Parameters

```
game-normal-form(
  p1: content,
  p2: content,
  s1: array,
  s2: array,
  payoffs,
  p1-best,
  p2-best,
  nash,
  cell-width: length,
  cell-height: length
) -> content
```

p1 `content`

Name of the row player, displayed rotated 90° on the left side of the matrix. Accepts any Typst content, e.g. [Alice] or [P_1].

p2 `content`

Name of the column player, displayed in bold above the column strategy labels. Accepts any Typst content.

s1 `array`

Array of strategy labels for Player 1, one element per row. The number of rows equals `s1.len()`. Each label is content and may include math or line breaks (e.g. [$p \backslash C$] for a two-line label).

s2 `array`

Array of strategy labels for Player 2, one element per column. The number of columns equals `s2.len()`. Column width auto-adjusts to the widest label in this array.

payoffs

2D array of payoff pairs indexed as `payoffs.at(row).at(col) = (v1, v2)`. Both `v1` (Player 1) and `v2` (Player 2) are Typst content and may contain arbitrary markup or mathematics. Example for a 2×2 game:

```
1 (
2   (([ $3$ ],[ $3$ ]), ([ $0$ ],[ $5$ ])),    // row 0
3   (([ $5$ ],[ $0$ ]), ([ $1$ ],[ $1$ ])),    // row 1
4 )
```

p1-best

Array of `(row, col)` tuples identifying cells where Player 1 plays a best response. Player 1's payoff in those cells is underlined in `game-pal.at(0)`.

Default: `()`

p2-best

Array of (row, col) tuples identifying cells where Player 2 plays a best response. Player 2's payoff in those cells is underlined in `game-pal.at(1)`.

Default: ()

nash

Array of (row, col) tuples of Nash equilibrium cells. A rectangle in `game-nash-color` is drawn just inside the cell border.

Default: ()

cell-width `length`

Minimum cell width. The actual width is the maximum of this value, the widest Player 2 strategy label width plus 2em, and the widest payoff pair width plus 2em. Increase to enforce a larger minimum regardless of content. Defaults to `game-cell-width`.

Default: `game-cell-width`

cell-height `length`

Fixed cell height. Content is placed at the cell centre; this value is not auto-sized, so increase it when payoffs span multiple lines. Defaults to `game-cell-height`.

Default: `game-cell-height`

game-three-player-normal-form

Draws a three-player normal form game as a collection of $N \times M$ payoff matrices, one per strategy of Player 3, laid out in rows of at most `games-per-row` matrices. All sub-matrices share the same cell dimensions. Payoffs for all three players are shown in each cell. Strategy labels of Player 3 appear above each sub-matrix.

Parameters

```
game-three-player-normal-form(  
  p1: content ,  
  p2: content ,  
  p3: content ,  
  s1: array ,  
  s2: array ,  
  s3: array ,  
  payoffs,  
  p1-best,  
  p2-best,  
  p3-best,  
  nash,  
  cell-width: length ,  
  cell-height: length ,  
  games-per-row: int  
) -> content
```

p1 content

Name of the row player (Player 1), displayed rotated 90° to the left of each row of sub-matrices.

p2 content

Name of the column player (Player 2), displayed in bold above the strategy labels of each sub-matrix.

p3 content

Name of the matrix player (Player 3), displayed once in bold above the first row of sub-matrices.

s1 array

Array of strategy labels for Player 1, shared across all sub-matrices. Determines the number of rows; `s1.len()` rows are drawn.

s2 array

Array of strategy labels for Player 2, shared across all sub-matrices. Determines the number of columns; `s2.len()` columns are drawn.

s3 **array**

Array of strategy labels for Player 3. One sub-matrix is drawn per entry, so `s3.len()` sub-matrices are produced in total.

payoffs

3D array of payoff triples indexed as `payoffs.at(k).at(r).at(c) = (v1, v2, v3)`. `k` is the Player 3 strategy index (outermost), `r` the row (Player 1), `c` the column (Player 2). All three values are Typst content.

p1-best

Array of `(k, row, col)` tuples where Player 1 plays a best response. Player 1's payoff is underlined in `game-pal.at(0)` at those cells.

Default: `()`

p2-best

Array of `(k, row, col)` tuples where Player 2 plays a best response. Player 2's payoff is underlined in `game-pal.at(1)` at those cells.

Default: `()`

p3-best

Array of `(k, row, col)` tuples where Player 3 plays a best response. Player 3's payoff is underlined in `game-pal.at(2)` at those cells.

Default: `()`

nash

Array of `(k, row, col)` tuples of Nash equilibrium cells. A rectangle in `game-nash-color` is drawn inside those cells.

Default: `()`

cell-width **length**

Minimum cell width, auto-grown to fit the widest payoff triple or Player 2 strategy label across all sub-matrices. Defaults to `game-cell-width`.

Default: `game-cell-width`

cell-height **length**

Fixed cell height. Increase when payoff content spans multiple lines. Defaults to `game-cell-height`.

Default: `game-cell-height`

games-per-row **int**

Maximum number of sub-matrices placed side by side. If Player 3 has more strategies than this, additional rows of sub-matrices are added below. A lone sub-matrix in the last row is horizontally centred. Defaults to `game-games-per-row`.

Default: `game-games-per-row`

game-pal

Per-player colour palette. Index 0 = Player 1, index 1 = Player 2, index 2 = Player 3. Defaults to `pi-player-colors` from `pi-game-palette`. Override after import: `#let game-pal = (rgb "..."), ...`.

game-nash-color

Colour for Nash equilibrium cell outlines. Defaults to `pi-nash-color` from `pi-game-palette`. Override after import: `#let game-nash-color = rgb "..."`.

game-fg

Foreground colour for cell borders, comma separators, and punctuation. Defaults to `pi-fg` from `pi-game-palette`. Override after import: `#let game-fg = rgb "..."`.

game-cell-width

Minimum cell width for payoff matrix columns [length]. The actual width is auto-grown to fit the widest payoff or strategy label. Override after import: `#let game-cell-width = 6em`.

game-cell-height

Fixed cell height for payoff matrix rows [length]. Override after import: `#let game-cell-height = 3em`.

game-games-per-row

Maximum number of sub-matrices placed side by side in three-player games [int].
Override after import: `#let game-games-per-row = 3`.